Python: exercises on datetime module

This notebook was generated in **solutions** mode.

Exercise 1: Age Calculator

Exercise Description

Write a function `calculate_age(birthdate)` that takes a date of birth (as a datetime object) and returns the person's age in years. Use the current date to calculate the age.

The function should properly handle cases where the birthday hasn't occurred yet this year.

The function should return the age as an integer.

Your solution

```python
from datetime import datetime

def calculate_age(birthdate):
    today = datetime.today()
    age = today.year - birthdate.year
    # Check if the birthday has occurred this year; if not, subtract one year
    if (today.month, today.day) < (birthdate.month, birthdate.day):
        age -= 1
    return age

# Test the function
dob = datetime(1990, 4, 25)
print(f"Age: {calculate_age(dob)} years")
```

Test your solution

```python
# Test the function with known birthdates
from datetime import datetime

# Test with a birthdate from 30 years ago (birthday already passed this year)
thirty_years_ago = datetime(datetime.now().year - 30, 1, 1)
assert calculate_age(thirty_years_ago) == 30

# Test with a birthdate from 25 years ago (birthday not yet passed this year)
twenty_five_years_ago = datetime(datetime.now().year - 25, 12, 31)
expected_age = 24 if datetime.now().month < 12 or (datetime.now().month == 12 and dat
assert calculate_age(twenty_five_years_ago) == expected_age
```

Test your solution

```python
# Test edge cases
from datetime import datetime

# Test with today's date (age should be 0)
today = datetime.today()
assert calculate_age(today) == 0

# Test with someone born exactly one year ago
one_year_ago = datetime(datetime.now().year - 1, datetime.now().month, datetime.now()
assert calculate_age(one_year_ago) == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: Days Until Event

Exercise Description

Write a function `days_until_event(event_date)` that takes a future date (as a datetime object) and calculates the number of days from today until that date.

The function should return the number of days as an integer. If the event date is in the past, the function should return a negative number.

Your solution

```
from datetime import datetime

def days_until_event(event_date):
    today = datetime.today()
    difference = event_date - today
    return difference.days

# Test the function
event = datetime(2024, 12, 25)
print(f"Days until event: {days_until_event(event)} days")
```

Test your solution

```
# Test the function with known future dates
from datetime import datetime, timedelta

# Test with a date 10 days from now
ten_days_future = datetime.today() + timedelta(days=10)
assert days_until_event(ten_days_future) == 10

# Test with a date 30 days from now
thirty_days_future = datetime.today() + timedelta(days=30)
assert days_until_event(thirty_days_future) == 30
```

Test your solution

```
# Test edge cases
from datetime import datetime, timedelta

# Test with today's date (should be 0)
today = datetime.today()
assert days_until_event(today) == 0

# Test with a past date (should be negative)
past_date = datetime.today() - timedelta(days=5)
assert days_until_event(past_date) == -5
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Date Range Generator

Exercise Description

Write a function `generate_date_range(start_date, end_date)` that takes two dates and returns a list of all the dates between them (inclusive). Use timedelta to increment each day.

The function should return a list of date strings in the format "YYYY-MM-DD".

Store the result in a variable called `result` so it can be tested.

Your solution

```
from datetime import datetime, timedelta

def generate_date_range(start_date, end_date):
    date_list = []
    current_date = start_date
    while current_date <= end_date:
        date_list.append(current_date.strftime("%Y-%m-%d"))
        current_date += timedelta(days=1)
    return date_list

# Test the function
start = datetime(2024, 9, 15)
end = datetime(2024, 9, 20)
result = generate_date_range(start, end)
for date_str in result:
    print(date_str)
```

Test your solution

```
# Test the function with the provided dates
from datetime import datetime

# Test with the example dates (September 15-20, 2024)
expected_dates = ["2024-09-15", "2024-09-16", "2024-09-17", "2024-09-18", "2024-09-19
assert result == expected_dates
assert len(result) == 6
```

Test your solution

```
# Test with additional date ranges
from datetime import datetime

# Test with a single day range
single_day_result = generate_date_range(datetime(2024, 1, 1), datetime(2024, 1, 1))
assert single_day_result == ["2024-01-01"]

# Test with a 3-day range
three_day_result = generate_date_range(datetime(2024, 12, 29), datetime(2024, 12, 31)
assert three_day_result == ["2024-12-29", "2024-12-30", "2024-12-31"]
assert len(three_day_result) == 3
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**